

Tag 4 - Wenn Fehler sich zusammenrotten: Bündelfehler, Interleaving und CRC

Der Hamming-Code von gestern ist großartig – aber er hat eine Schwäche: zwei oder mehr benachbarte Fehler werfen ihn aus der Bahn. Genau das passiert aber in der echten Welt: zerkratzte CDs, dreckige Etiketten, gestörte WLAN-Pakete. Heute lernen wir, wie man damit umgeht – und treffen auf zwei neue Ideen, die in fast jedem digitalen Gerät stecken: **Interleaving** und **CRC**.

Lernziele für heute

Am Ende von Tag 4 kannst du ...

- erklären, warum **Bündelfehler** (mehrere benachbarte Bits) in der Praxis viel häufiger sind als zufällig verteilte Einzelfehler
- den Hamming-Code zu **SECDED** erweitern und damit Doppelfehler **erkennen** (auch wenn nicht korrigieren)
- mit **Interleaving** einen Bündelfehler in lauter Einzelfehler verwandeln
- ein **CRC** (Cyclic Redundancy Check) berechnen und prüfen
- den Unterschied zwischen *Erkennen* und *Korrigieren* in der Praxis abschätzen und entscheiden, welches Verfahren zu welcher Anwendung passt

Material: Bleistift, kariertes Papier, Thonny.

Block 1 · Warum Bündelfehler so gemein sind (≈ 30 min)

Stell dir folgende Situation vor: Du brennst eine CD, machst einen Kratzer rein und versuchst, sie zu lesen. Der Kratzer ist 2 mm lang. Auf der CD-Oberfläche sind das, sagen wir mal, 200 Bits hintereinander, die nicht mehr gelesen werden können. **200 benachbarte Bitfehler.**

Oder ein anderes Beispiel: Funkverbindung. Wenn ein Mobilfunk-Sender kurz von einem Auto verdeckt wird (sogenanntes *Fading*), gehen *alle* Bits, die in diesem Moment ankommen, verloren. Wieder ein Bündel – diesmal vielleicht Hunderte oder Tausende von Bits.

Solche **Bündelfehler** (engl. *burst errors*) sind in der Realität die Regel, nicht die Ausnahme.

Wie verhält sich Hamming bei einem Bündelfehler?

Erinnere dich an deinen (7,4)-Hamming-Code von gestern. Er korrigiert genau einen Bitfehler.

- (a) Was passiert, wenn auf einem Codewort **zwei** benachbarte Bits gleichzeitig kippen? Du hast das gestern in Aufgabe 3.1 schon ausprobiert. Was war das Ergebnis?



Bleistift-
Übung 1

- (b) Stell dir vor, ein Bündelfehler über **3** benachbarte Bits trifft dein Codewort. Was glaubst du, ergibt der Decoder?
- (c) Diskutiert: Wäre es eine gute Idee, einfach Hamming-Codes hintereinanderschalten? Wenn ja, warum? Wenn nein, warum nicht?

Tipp zu (c): Wenn du z.,B. 4 Hamming-Codewörter hintereinander schreibst – also $7+7+7+7 = 28$ Bits – und dann ein Bündelfehler über 3 Bits genau zwischen Wort 2 und Wort 3 hereinrutscht: in welchen Wörtern sind die Fehler? Wie viele Fehler pro Wort?

Eine erste Idee: SECDED

Bevor wir Bündelfehler richtig angehen, machen wir noch eine kleine Erweiterung des Hamming-Codes, die Doppelfehler immerhin **erkennt**, auch wenn sie sie nicht korrigieren kann.

Idee: Wir hängen an den (7,4)-Hamming-Code ein **achtes Bit** an – die Gesamt-Parität über alle 7 Bits. Aus dem (7,4)-Code wird der **(8,4)-SECDED-Code** (Single Error Correction, Double Error Detection).

So funktioniert das beim Empfänger:

Hamming-Prüfung	Gesamt-Parität	Diagnose
ok	ok	kein Fehler
Fehler erkannt	falsch	1 Fehler → korrigierbar
Fehler erkannt	ok	2 Fehler → erkannt, aber unrettbar
ok	falsch	nur das achte Bit selbst gekippt

Der Trick steckt in der **Kombination**: Bei einem Einzelfehler ist sowohl die Hamming-Prüfung als auch die Gesamt-Parität gestört (denn 1 ist eine ungerade Anzahl). Bei einem Doppelfehler ist die Hamming-Prüfung gestört, aber die Gesamt-Parität bleibt richtig (denn 2 ist gerade). Daraus lässt sich eindeutig unterscheiden, ob ein Einzel- oder ein Doppelfehler vorlag!

SECDED nachvollziehen

Codewort von gestern: 0 1 1 0 0 1 1 (für Datenwort 1 0 1 1).

- (a) Berechne das Gesamt-Paritätsbit P, sodass die Anzahl der Einsen im erweiterten Codewort **gerade** ist. Schreibe das vollständige (8,4)-Codewort auf.
- (b) Du empfangst 0 1 0 0 0 1 1 0 (achtes Bit ist deine berechnete Gesamt-Parität, das vorletzte Bit wurde gekippt). Berechne die Hamming-Prüfsummen s_1, s_2, s_3 . Berechne die Gesamt-Parität. Welcher Fall aus der Tabelle liegt vor?
- (c) Empfangen: 1 1 0 0 0 0 1 0. Welcher Fall liegt vor?



Bleistift-
Übung 2

SECDED steckt heute in jedem **ECC-RAM** in Servern. Das gestern angesprochene 64-Bit-Wort mit 8 Prüfbits ist genau ein SECDED-Code: er korrigiert 1 Bitfehler pro 64-Bit-Wort und erkennt 2 Bitfehler.

Block 2 · Interleaving - der einfachste Trick gegen Bündelfehler (≈ 45 min)

SECDED ist nett, aber Bündelfehler sind damit immer noch nicht im Griff – wir können einen Doppelfehler ja nur **erkennen**, nicht korrigieren. Was tun, wenn zerkratzte CDs ganze 200-Bit-Bündel zerstören?

Die Idee von Interleaving: Verteile die Daten so über die Übertragung, dass benachbarte Bits in der Übertragung *nicht* benachbart in den Daten sind. Wenn dann ein Bündelfehler kommt, betrifft er aus jedem Codewort nur **ein einziges Bit** – und das kann der Hamming-Code wieder korrigieren.

Das Bild dazu

Stell dir vor, du hast 4 Hamming-Codewörter A, B, C, D, jedes 7 Bit lang:

A: a1 a2 a3 a4 a5 a6 a7
B: b1 b2 b3 b4 b5 b6 b7
C: c1 c2 c3 c4 c5 c6 c7
D: d1 d2 d3 d4 d5 d6 d7

Ohne Interleaving würden wir die einfach hintereinander schreiben:

a1 a2 a3 a4 a5 a6 a7 | b1 b2 b3 b4 b5 b6 b7 | c1 ... | d1 ...

Ein Bündelfehler über 4 Bits in der Mitte trifft alle 4 Bits desselben Codeworts → unrettbar.

Mit Interleaving schreiben wir spaltenweise:

a1 b1 c1 d1 | a2 b2 c2 d2 | a3 b3 c3 d3 | ... | a7 b7 c7 d7

Ein Bündelfehler über 4 Bits trifft jetzt aus jedem Codewort genau **1 Bit**. Jeder Hamming-Code-Decoder kann das korrigieren!

Interleaving auf Karopapier

Hier sind 4 sehr kurze Codewörter (aus einem fiktiven 4-Bit-Code):



A: 1 0 1 0
 B: 0 1 1 1
 C: 1 1 0 0
 D: 0 0 1 1

- Schreibe sie hintereinander auf (ohne Interleaving). Wie lang ist die resultierende Zeile?
- Schreibe sie nun in eine 4×4-Tabelle (eine Zeile pro Codewort) und lies die Daten **spaltenweise** aus. Schreibe die so entstandene Bitfolge auf.
- Nimm an, ein Bündelfehler kippt in der interleavten Folge die Bits an Position 5, 6, 7, 8. Schreibe die gestörte Folge auf, baue sie zurück in die 4×4-Tabelle und prüfe: Wie viele Fehler hat jedes Codewort jetzt?
- Würde derselbe 4-Bit-Bündelfehler ohne Interleaving (Variante a) ein Codewort komplett ruinieren?

Interleaver bauen



Python-
Einheit 1

Lege eine neue Datei interleaver.py an:

```
def interleave(codewoerter):
    """Nimmt eine Liste von gleich langen Codewörtern und liefert
    die spaltenweise interleaved Bitfolge."""
    n = len(codewoerter[0])
    ergebnis = []
    for spalte in range(n):
        for codewort in codewoerter:
            ergebnis.append(codewort[spalte])
    return ergebnis

def deinterleave(bitfolge, anzahl_codewoerter):
    """Kehrt das Interleaving um."""
    n = len(bitfolge) // anzahl_codewoerter
    codewoerter = [[] for _ in range(anzahl_codewoerter)]
    idx = 0
    for spalte in range(n):
        for i in range(anzahl_codewoerter):
            codewoerter[i].append(bitfolge[idx])
            idx += 1
    return codewoerter

# Test:
codes = [[1,0,1,0], [0,1,1,1], [1,1,0,0], [0,0,1,1]]
print("Codes:", codes)
verschachtelt = interleave(codes)
print("Interleaved:", verschachtelt)
zurueck = deinterleave(verschachtelt, 4)
print("Deinterleaved:", zurueck)
print("Identisch?", zurueck == codes)
```

Aufgabe 1.1 - Bündelfehler simulieren Nutze deinen hamming74_encode/hamming74_decode aus Tag 3 zusammen mit dem Interleaver. Probiere folgendes:

1. Wähle 4 Datenwörter (je 4 Bits). Codiere jedes mit Hamming → 4 Codewörter à 7 Bit.
2. Interleave die 4 Codewörter zu einer 28-Bit-Folge.
3. Simuliere einen Bündelfehler: kippe **4 benachbarte Bits** in dieser Folge.
4. Deinterleave zurück in 4 Codewörter à 7 Bit.
5. Decodiere jedes Codewort einzeln mit `hamming74_decode`.
6. Prüfe: Sind alle 4 Datenwörter korrekt rekonstruiert worden?

Erwartung: Bei 4 Codewörtern und 4 Bündelfehler-Bits hat jedes Codewort *höchstens 1 Fehler* → Hamming kann jeden korrigieren → alle Datenwörter werden gerettet.

Die Praxis: CD und Compact-Disc-DA

Auf einer Audio-CD steckt diese Idee in massiv ausgebauter Form: **Cross-Interleaved Reed-Solomon Code (CIRC)**. Die Audio-Daten werden über *Hunderte* Symbole interleaved, sodass selbst Kratzer von mehreren Millimetern Länge problemlos rekonstruiert werden können – aus benachbarten Symbolen in der Übertragung werden weit entfernte Symbole in den Originaldaten. Wenn du heute auf Spotify einen alten Song hörst, der von einer CD-Master-Tape kam: dieser CIRC hat ihn überlebt.

Block 3 · CRC - die Praxis-Standardlösung für Erkennung (≈ 45 min)

Bisher hatten wir es mit *Korrektur* zu tun. Aber in vielen Anwendungen reicht es, wenn man Fehler **erkennt** und das fehlerhafte Paket einfach **erneut anfordert**:

- Bei Ethernet (Netzwerkkarten): wenn ein Paket fehlerhaft ankommt, wird es einfach noch einmal verlangt.
- Bei TCP (das, was hinter `https://` und SSH und fast allem im Internet steckt): wenn ein Paket fehlt oder kaputt ist, wird es nochmal geschickt.
- Bei ZIP-Dateien: wenn das CRC nicht stimmt, weiß das Archivierungsprogramm, dass die Datei kaputt ist.

In all diesen Fällen ist das Werkzeug der Wahl: **CRC** (Cyclic Redundancy Check).

Die Idee von CRC

Bei einer Prüfziffer wie EAN-13 hatten wir das Datenwort als Zahl betrachtet und modulo eine Primzahl gerechnet. CRC macht dasselbe – nur mit **Polynomen statt Zahlen**.

Dazu betrachten wir eine Bitfolge wie 1 0 1 1 0 1 als Polynom:

$$1 \cdot x^5 + 0 \cdot x^4 + 1 \cdot x^3 + 1 \cdot x^2 + 0 \cdot x^1 + 1 \cdot x^0 \\ = x^5 + x^3 + x^2 + 1$$

Jedes Bit ist ein Koeffizient (entweder 0 oder 1) vor einer Potenz von x .

CRC-3 (winzig, aber aussagekräftig)

Das einfachste sinnvolle CRC: wir wählen ein **Generatorpolynom** mit Grad 3, zum Beispiel:

$$G(x) = x^3 + x + 1 \rightarrow \text{binär: } 1011$$

Das Verfahren:

1. Hänge an die Daten (k Bit) drei Nullen an.
2. Teile die so entstandene Zahl durch $G(x)$, aber: **Modulo 2 bei jedem Schritt** (das heißt, statt Subtraktion machen wir XOR – kein Übertrag, kein Vorzeichen).
3. Der **Rest** der Division ist das CRC. Es hat genau 3 Bits (denn der Grad von G ist 3).
4. Hänge den Rest an die Daten an. Das ist das gesendete Wort.

CRC von Hand

Daten: 1 0 1 1 0 (5 Bits).

(a) Hänge 3 Nullen an: 1 0 1 1 0 0 0 0 (8 Bits).

(b) Führe die Polynomdivision modulo 2 durch. Das geht wie schriftliche Division im Kopf, nur mit XOR statt Subtraktion. Hier zur Hilfe der Anfang:

$$\begin{array}{r} 10110000 \div 1011 \\ 1011 \\ \hline 00000000 \oplus \\ | \\ \text{ ab hier weitermachen ...} \end{array}$$

Tipp: XOR ist $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$. Also: gleiche Bits $\rightarrow 0$, verschiedene Bits $\rightarrow 1$.

(c) Welcher Rest bleibt nach der Division übrig? Das sind die 3 CRC-Bits.

(d) Schreibe das gesendete Wort auf: 5 Daten-Bits + 3 CRC-Bits = 8 Bits.

(e) Beim Empfänger: Die ganzen 8 Bits werden durch dasselbe $G(x)$ geteilt. Wenn der Rest **0** ist, war alles in Ordnung. Probiere es: Teile dein Ergebnis aus (d) durch 1011. Was bleibt übrig?

CRC implementieren



Bleistift-
Übung 4



Python-
Einheit 2

```

def crc_berechnen(daten_bits, generator_bits):
    """Berechnet das CRC für eine Liste von Datenbits.
    daten_bits: Liste der Daten (z.B. [1,0,1,1,0])
    generator_bits: Liste der Generator-Bits (z.B. [1,0,1,1] für  $G(x) = x^3+x+1$ )
    Rückgabe: Liste der CRC-Bits (Länge =  $\text{len}(\text{generator\_bits}) - 1$ )"""
    grad = len(generator_bits) - 1
    # Daten plus Nullen anhängen
    arbeit = list(daten_bits) + [0] * grad

    # Polynomdivision modulo 2
    for i in range(len(daten_bits)):
        if arbeit[i] == 1: # nur teilen, wenn führendes Bit 1 ist
            for j in range(len(generator_bits)):
                arbeit[i + j] ^= generator_bits[j] # XOR

    # Die letzten 'grad' Bits sind der Rest = CRC
    return arbeit[len(daten_bits):]

def crc_pruefen(empfangen_bits, generator_bits):
    """Prüft, ob die empfangene Folge (Daten + CRC) durch G teilbar ist.
    Rückgabe: True wenn CRC stimmt, False sonst."""
    grad = len(generator_bits) - 1
    daten = empfangen_bits[:-grad]
    erwartet = crc_berechnen(daten, generator_bits)
    return erwartet == empfangen_bits[-grad:]

# Test:
generator = [1, 0, 1, 1] #  $G(x) = x^3 + x + 1$ 
daten = [1, 0, 1, 1, 0]
crc = crc_berechnen(daten, generator)
print("CRC:", crc)

gesendet = daten + crc
print("Gesendet:", gesendet)
print("Empfangen ok?", crc_pruefen(gesendet, generator))

# Mit Fehler:
gestoert = gesendet.copy()
gestoert[2] = 1 - gestoert[2]
print("Empfangen ok (mit Fehler)?", crc_pruefen(gestoert, generator))

```

Aufgabe 2.1 - Wie viele Fehler erkennt CRC-3? Schreibe einen Brute-Force-Test: Für ein gültiges CRC-Wort der Länge 8 probiere **alle 256 möglichen Fehler-Muster** (jedes Bit kann gekippt sein oder nicht) und zähle, wie viele davon erkannt werden.

```

import itertools

def teste_alle_fehler(gesendet, generator):
    n = len(gesendet)
    erkannt = 0
    nicht_erkannt = 0
    nicht_erkannte_muster = []
    # Alle möglichen Fehler-Muster (außer dem Null-Muster, das ist ja kein Fehler)
    for fehler_muster in itertools.product([0,1], repeat=n):

```

```

if all(b == 0 for b in fehler_muster):
    continue # Null-Muster überspringen
# Fehler einbauen
gestoert = [g ^ f for g, f in zip(gesendet, fehler_muster)]
if crc_pruefen(gestoert, generator):
    nicht_erkannt += 1
    nicht_erkannte_muster.append(fehler_muster)
else:
    erkannt += 1
return erkannt, nicht_erkannt, nicht_erkannte_muster

```

```

erkannt, nicht_erkannt, muster = teste_alle_fehler(gesendet, generator)
print(f"{erkannt} erkannt, {nicht_erkannt} nicht erkannt")
print(f"Nicht erkannte Muster (Anzahl): {len(muster)}")

```

Aufgabe 2.2 - Welche Fehler bleiben unentdeckt? Bei einem CRC mit r Prüfbits gibt es immer genau 2^r mögliche Fehler-Muster, die unentdeckt bleiben (auch das Null-Muster, das aber kein Fehler ist). Bei CRC-3 ($r=3$) sind das $2^3 = 8$ Muster (inkl. Null). Schau dir die nicht erkannten Muster an: Was ist die kleinste Anzahl von gekippten Bits, die unentdeckt bleibt?

Spoiler: Bei einem gut gewählten Generatorpolynom werden alle 1-Bit- und 2-Bit-Fehler erkannt (für CRC-3 zumindest in einer bestimmten Maximallänge). Erst ab 3 Bit Fehlern wird's interessant.

CRC in der Praxis

Die wirklich wichtigen CRCs sind:

Name	Grad	Bekannt aus
CRC-8	8	Bluetooth, ATM-Netze
CRC-16-CCITT	16	Bluetooth, X.25, HDLC
CRC-32	32	Ethernet, ZIP-Dateien, PNG, MPEG
CRC-32C	32	iSCSI, ext4

CRC-32 erkennt mit hoher Wahrscheinlichkeit (besser als 1 zu 4 Milliarden) jeden Bündelfehler bis 32 Bit Länge und alle Doppelfehler in Paketen bis ca. 8 KB. Genau deshalb ist es der De-facto-Standard im Internet-Stack.

Block 4 · Reflexion und Brücke (≈ 20 min)

Was wir heute gelernt haben

- **Bündelfehler** sind in der Realität die Regel, nicht die Ausnahme – Kratzer, Funkfading, defekte Speichersektoren betreffen immer mehrere benachbarte Bits.

- **SECCDED** erweitert Hamming um ein zusätzliches Paritätsbit. Es kann immer noch nur 1 Fehler korrigieren, erkennt aber sicher 2.
- **Interleaving** ist eine geniale, einfache Idee: Daten so verschachteln, dass Bündelfehler zu Einzelfehlern werden, die der innere Code (z. B. Hamming) korrigieren kann. Steckt in jeder CD, DVD und vielen Funkstandards.
- **CRC** ist das Standardverfahren für Erkennung (ohne Korrektur). Es nutzt Polynomdivision modulo 2 und ist mathematisch eng mit dem verwandt, was wir noch kennenlernen werden.

Drei Fragen zum Mitnehmen

1. CRC erkennt mehr Fehlertypen als eine einfache Parität, kostet aber auch mehr Prüfbits. Wenn du auf einem stark gestörten Funkkanal arbeitest – würdest du eher CRC einsetzen, oder einen fehlerkorrigierenden Code? Was ist der Tradeoff?
2. Du hast bei CRC bemerkt: Wir rechnen mit **Polynomen modulo 2**. Bei EAN-13 haben wir mit Zahlen modulo 10 gerechnet, bei ISBN-10 mit Zahlen modulo 11. Sieht aus, als wäre Modulo *die* universelle Idee. Wie könnte man das auf Bytes (also Zahlen 0..255) übertragen?
3. Beim Hamming-Code mussten wir genau wissen, *welche* Bits gekippt sind, um zu korrigieren. Bei CRC interessiert uns nur, *ob* irgendwas gekippt ist. Welcher Ansatz braucht weniger Information – und warum funktioniert er trotzdem?

Vorschau Tag 5

Frage 2 oben ist nicht zufällig gestellt. Morgen verlassen wir endgültig die Welt der Bits und steigen in die Welt der **Bytes** ein – nicht als 8 Bits, sondern als eigenständige Zahlen 0 bis 255. Dafür brauchen wir eine neue Mathematik: **endliche Körper** (oder *Galois-Felder*, abgekürzt GF). Klingt fies, ist aber im Grunde nur „Modulo-Arithmetik mit Polynomen statt Zahlen“ – und wir haben heute mit CRC schon den Fuß in der Tür gehabt.

Mit endlichen Körpern können wir dann verstehen, wie **Reed-Solomon** funktioniert – und damit sind wir nur noch wenige Schritte vom **Datamatrix** entfernt.

Lösungen (erst nach eigenem Versuch ansehen!)

Bleistiftübung 1 - Hamming bei Bündelfehlern

- (a) Bei zwei benachbarten Fehlern wird der (7,4)-Hamming-Code immer **falsch korrigieren** - er denkt, es war ein Einzelfehler an einer dritten, falschen Position, und kippt dort ein Bit. Resultat: 3 Bits falsch, Datenwort fast immer kaputt.
- (b) Bei 3 Bündelfehlern: ähnlich, der Decoder produziert Müll. Mit jeder weiteren Fehlerzahl wird das Ergebnis fast unkorreliert mit dem Original.
- (c) Hintereinanderschalten von Hamming-Codes löst das Problem **nicht**. Schauen wir uns das genauer an: 4 Codewörter à 7 Bit ergeben 28 Bits. Ein Bündelfehler von 3 Bits zwischen Wort 2 und Wort 3 trifft zum Beispiel die letzten 2 Bits von Wort 2 und das erste Bit von Wort 3. Wort 2 hat also 2 Fehler → unrettbar. Hintereinanderschalten reicht nicht; wir brauchen **Interleaving**.

Bleistiftübung 2 - SECDED

- (a) Codewort 0 1 1 0 0 1 1 hat 4 Einsen → Anzahl ist gerade → $P = 0$. Vollständiges (8,4)-Codewort: 0 1 1 0 0 1 1 0.
- (b) Empfangen: 0 1 0 0 0 1 1 0 (Bit 3 ist gekippt verglichen mit dem Original).

Hamming-Prüfsummen:

- $s_1 = b_1 + b_3 + b_5 + b_7 \bmod 2 = 0 + 0 + 0 + 1 = 1$
- $s_2 = b_2 + b_3 + b_6 + b_7 \bmod 2 = 1 + 0 + 1 + 1 = 3 \bmod 2 = 1$
- $s_3 = b_4 + b_5 + b_6 + b_7 \bmod 2 = 0 + 0 + 1 + 1 = 2 \bmod 2 = 0$

$s_3 s_2 s_1 = 011$ binär = 3 → Hamming meldet Fehler an Position 3.

Gesamt-Parität des Empfangen: $0 + 1 + 0 + 0 + 0 + 1 + 1 + 0 = 3$ (ungerade) → Gesamt-Parität ist falsch.

Hamming-Prüfung **Fehler erkannt** + Gesamt-Parität **falsch** → **1 Fehler, korrigierbar** an Position 3. □

- (c) Empfangen: 1 1 0 0 0 0 1 0. Verglichen mit 0 1 1 0 0 1 1 0 sind Bits 1, 3, 6 gekippt - 3 Fehler. Aber wir tun so, als wüssten wir das nicht.

Hamming-Prüfsummen:

- $s_1 = 1 + 0 + 0 + 1 = 2 \bmod 2 = 0$
- $s_2 = 1 + 0 + 0 + 1 = 2 \bmod 2 = 0$
- $s_3 = 0 + 0 + 0 + 1 = 1$

$s_3 s_2 s_1 = 100$ binär = 4 → Hamming meldet Fehler an Position 4.

Gesamt-Parität: $1+1+0+0+0+0+1+0 = 3$ (ungerade) → falsch.

Hamming-Prüfung **Fehler erkannt** + Gesamt-Parität **falsch** → SECDED denkt: 1 Fehler an Position 4. **Aber das ist falsch!** Tatsächlich waren es 3 Fehler. SECDED korrigiert hier zur falschen Antwort.

SECDED garantiert: 1 Fehler korrigiert, 2 Fehler erkannt, 3 oder mehr Fehler – keine Garantie. Genau wie Hamming-Code: jenseits der Garantiegrenze wird's gefährlich.

Bleistiftübung 3 - Interleaving auf Karopapier

(a) Hintereinander: 1010 0111 1100 0011. Das sind 16 Bits.

(b) Tabelle und spaltenweise lesen:

	Sp1	Sp2	Sp3	Sp4
A:	1	0	1	0
B:	0	1	1	1
C:	1	1	0	0
D:	0	0	1	1

Spaltenweise: 1010 0110 1101 0101. (Spalte 1 = 1,0,1,0, Spalte 2 = 0,1,1,0, ...)

(c) Bündelfehler kippt Bits an Position 5–8: das ist Spalte 2, also je das zweite Bit jedes Codeworts.

Original Spalte 2: 0,1,1,0. Gestört: 1,0,0,1.

Zurück in Tabelle:

A:	1	1	1	0	(vorher 1010)	→ 1 Fehler in Position 2
B:	0	0	1	1	(vorher 0111)	→ 1 Fehler in Position 2
C:	1	0	0	0	(vorher 1100)	→ 1 Fehler in Position 2
D:	0	1	1	1	(vorher 0011)	→ 1 Fehler in Position 2

Jedes Codewort hat genau **1 Fehler**. Wenn die Codewörter Hamming-codiert wären, könnte jeder Decoder seinen eigenen Einzelfehler korrigieren!

(d) Ohne Interleaving würde der 4-Bit-Bündelfehler an Position 5–8 das gesamte Codewort B zerstören (4 Fehler in B, 0 in A/C/D). Hamming kann B nicht retten.

Aufgabe 1.1 (Python) - Interleaving-Test

Erwartete Ausgabe: alle 4 Datenwörter werden korrekt rekonstruiert. Wenn du den Bündelfehler auf 5 Bits oder mehr erweiterst, könnte ein Codewort zwei Fehler bekommen → dann scheitert die Korrektur. Ab 8 Bündelfehler wird es definitiv kritisch.

Bleistiftübung 4 - CRC

(a) 10110000 (5 Datenbits + 3 angehängte Nullen).

(b) Polynomdivision mod 2:

```

1 0 1 1 0 0 0 0
1 0 1 1           ← G·1
-----
0 0 0 0 0 0 0 0
    (führende 0en überspringen, weiter unten)

1 0 1 1 0 0 0 0
1 0 1 1
-----
0 1 1 0 0 0
    (führendes 0, eine Stelle weiter)

1 1 0 0 0
1 0 1 1           ← G·1 (an passender Stelle)
-----
0 1 1 1 0
    1 1 1 0
    1 0 1 1           ← G·1
    -----
        1 0 1

```

Rest: 1 0 1.

(c) CRC = 1 0 1. Vollständig gesendetes Wort: 1 0 1 1 0 1 0 1.

(d) Wenn man 10110101 durch 1011 teilt, sollte der Rest **0** sein – das ist die Eigenschaft, die zur CRC-Prüfung genutzt wird.

(Hinweis: Das ist auch der Trick, warum man auf der Senderseite die 000 anhängt: damit der Rest beim Empfänger genau dann 0 ist, wenn nichts gestört wurde.)

Aufgabe 2.1/2.2 - Brute-Force-Test

Erwartete Ausgabe (sinngemäß):

248 erkannt, 7 nicht erkannt

Nicht erkannte Muster (Anzahl): 7

Die 7 nicht erkannten Muster sind genau die Vielfachen von $G(x) = 1011$ selbst (außer dem Null-Muster, das ja gar kein Fehler ist):

00001011, 00010110, 00101100, 01011000, 10110000,
00010101, ... (etc.)

Allgemein gilt: ein CRC mit r Prüfbits hat genau $2^{(n-r)}$ Vielfache von $G(x)$ im n -Bit-Raum, also $2^{(n-r)} - 1$ nicht-triviale unentdeckte Fehlermuster.

Bei guter Wahl von $G(x)$: keine 1-Bit-Fehler unentdeckt, keine 2-Bit-Fehler bis zur Maximallänge unentdeckt – erst ab 3 Fehlerbits oder sehr großen Lücken werden überhaupt einige Muster nicht erkannt.

Antworten zu den drei Reflexionsfragen

1. **Korrektur vs. Erkennung:** Bei stark gestörten Kanälen ist Korrektur essenziell, weil Wiederholung teuer oder unmöglich ist (denk an Voyager – ein Re-Send dauert Stunden). In LANs/Internet ist Erkennung + Re-Send fast immer billiger. Die Faustregel: je teurer ein Re-Send, desto mehr Korrektur. Mobilfunk ist ein Mischmasch (Forward Error Correction für Schnelligkeit, plus Re-Send-Mechanismen oben drauf).
2. **Modulo für Bytes:** Genau das bereiten wir morgen vor. Mit Bytes (0–255) braucht man eine andere Modulo-Arithmetik. Naheliegend wäre Modulo 256 – aber das hat schlechte mathematische Eigenschaften (256 ist nicht prim und 2 hat dort keinen Inversen). Stattdessen nutzt man **Polynomarithmetik modulo eines irreduziblen Polynoms** über $GF(2)$. Klingt furchterregend, ist aber konzeptionell genau wie ISBN-10 modulo 11 – nur eben mit Polynomen.
3. **Mehr/weniger Information:** CRC braucht weniger Information, weil es nur „ja/nein“-Entscheidungen treffen muss. Korrektur braucht mehr, weil es zusätzlich die Position des Fehlers identifizieren muss. Daraus folgt: Erkennung kann mit weniger Redundanz erreicht werden als Korrektur – wenn man Re-Sends akzeptieren kann.